



PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG THÔNG TIN

Chương 4: Phân tích yêu cầu

ThS. Nguyễn Văn Hữu Hoàng

I NỘI DUNG

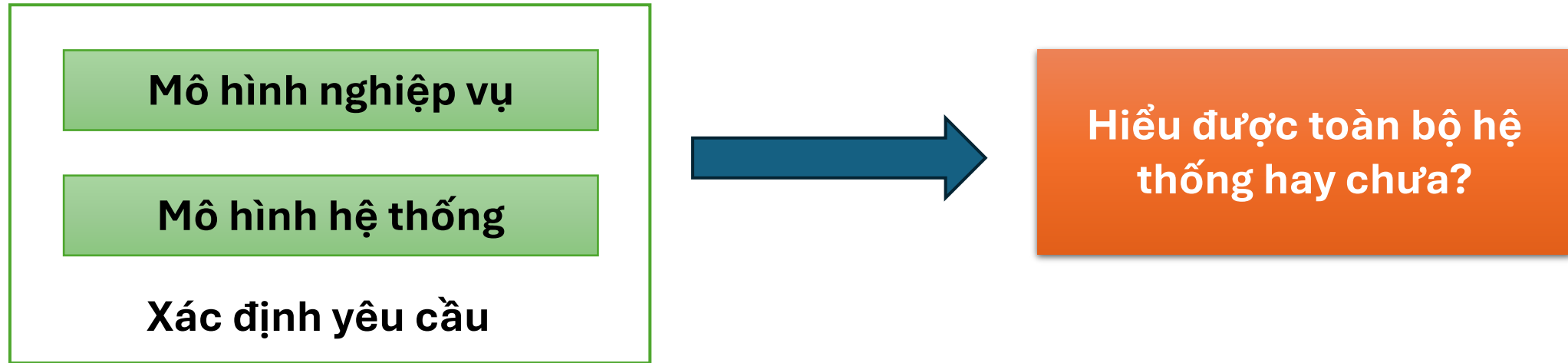
Chương này tập trung trình bày một số chủ đề sau đây:

- 4.1 Các bước trong pha phân tích yêu cầu
- 4.2. Xây dựng mô hình phân tích tĩnh
- 4.3. Xây dựng mô hình phân tích động

I MỤC ĐÍCH CỦA PHÂN TÍCH YÊU CẦU

- Phân tích yêu cầu một hệ thống là nhằm trả lời câu hỏi hệ thống **sẽ làm những gì**, hơn là chỉ ra cách thức làm những việc đó.
- Phân tích là pha giúp chúng ta hiểu được mô hình thực sự của hệ thống là gì.
- Quá trình phân tích yêu cầu thường được chia thành hai giai đoạn:
 - **Xây dựng mô hình phân tích tĩnh**: Mô hình phân tích tĩnh được thể hiện bởi một biểu đồ lớp để chỉ ra các đối tượng mà hệ thống sẽ sử dụng và cách thức chúng tương tác với nhau.
 - **Xây dựng mô hình phân tích động**: Mô hình phân tích động có thể được thể hiện bởi biểu đồ tương tác như biểu đồ giao tiếp hay biểu đồ tuần tự nhằm chỉ ra tính khả thi của mô hình phân tích tĩnh.
- Những kết quả ở pha *Xác định yêu cầu* cần phải được chuyển thành mô hình các đối tượng cùng với các thuộc tính và các mối quan hệ giữa chúng trong hệ thống. Những đối tượng này sẽ tồn tại bên trong chính hệ thống hoặc tại biên của hệ thống và có thể truy nhập được thông qua một hay nhiều giao diện.

| 4.1 Các bước trong pha phân tích yêu cầu



- **Mô hình nghiệp vụ**: chỉ mô tả thực tại của tổ chức mà chưa nói gì được hiện thực khi phần mềm được thêm vào.
 - **Mô hình hệ thống**: các ca sử dụng chỉ mô tả tương tác giữa tác nhân và biên của hệ thống nên hệ thống chỉ là một hộp đen khi nhìn từ bên ngoài.
- ➔ Cần phân tích kỹ hơn về các đối tượng nghiệp vụ.

| Các bước trong pha phân tích yêu cầu

- **Phân tích tĩnh**

1. Xác định các lớp
2. Xác định quan hệ giữa các lớp
3. Xây dựng biểu đồ lớp
4. Xác định thuộc tính lớp
5. Cập nhật bảng thuật ngữ và các yêu cầu phi chức năng (Tùy chọn)

- **Phân tích động**

1. Xây dựng biểu đồ giao tiếp (pha phân tích)
2. Gán phương thức cho các lớp

| Các bước trong pha phân tích yêu cầu

- Thuật ngữ hiện thực hóa ca sử dụng (**use case realization**) có nghĩa là làm cho các ca sử dụng trở thành cài đặt được. Những thao tác được khám phá trong suốt quá trình hiện thực hóa ca sử dụng có thể sẽ bị bỏ qua trong quá trình thiết kế. Vì vậy, trong giai đoạn này, chúng ta cố gắng xây dựng sự tin cậy về tính khả thi mà không thiết kế giải pháp.
- Chúng ta cần cho khách hàng xem các biểu đồ lớp với đầy đủ thuộc tính để họ giúp phát hiện lỗi vì KH chắc chắn hiểu biết về nghiệp vụ hơn chúng ta.
- Các biểu đồ giao tiếp và các thao tác của đối tượng không nên đưa cho khách hàng xem vì nó hơi phức tạp và thật ra là không cần thiết.

| 4.2 Phân tích tĩnh

- Mô hình phân tích tĩnh liên quan đến việc phân rã hệ thống thành các thành phần logic và vật lý và cách mà các thành phần này tương tác với nhau.
- Mục đích của nó là nhằm mô tả cách chúng ta xây dựng và khởi tạo hệ thống như thế nào.

| 4.2.1 Xác định các lớp

Định nghĩa các khái niệm

- **Đối tượng (tin học)** là một biểu diễn trừu tượng của một thực thể (vật lý hay khái niệm) có định danh và ranh giới rõ ràng trong thế giới thực, bao gồm cả trạng thái và hành vi của thực thể đó, nhằm mục đích mô phỏng hay điều khiển thực thể đó.
 - **Trạng thái** của đối tượng thể hiện bởi một tập hợp các thuộc tính. Ở mỗi thời điểm, mỗi thuộc tính của đối tượng có một giá trị nhất định.
 - **Hành vi** của đối tượng thể hiện bằng một tập hợp các thao tác, đó là các dịch vụ mà nó có thể thực hiện khi được một đối tượng khác yêu cầu.
 - **Định danh** của đối tượng là cái để phân biệt nó với đối tượng khác.
- **Lớp** là một mô tả của một tập hợp các đối tượng cùng có chung các thuộc tính, các thao tác, các mối liên quan, các ràng buộc và ngữ nghĩa.
- **Lớp** là một kiểu, và mỗi **đối tượng** thuộc lớp là một cá thể (instance).

| Biểu diễn lớp và đối tượng

- Biểu diễn lớp

Lớp
thuộc tính
thao tác

Lớp

Lớp

- Biểu diễn đối tượng

<u>đối tượng</u> :Lớp
thuộc tính = giá trị

<u>đối tượng</u>

<u>:Lớp</u>

| 4.2.1 Xác định các lớp

- Hầu như không có một quy tắc chung cho việc phát hiện ra các lớp
➔Đòi hỏi kỹ năng và kinh nghiệm của người phân tích.
- Danh sách lớp sẽ thay đổi theo thời gian vì tiến trình phân tích và thiết kế là tiến trình lặp đi lặp lại.
- Người ta thường hay sử dụng khái niệm các **lớp ứng viên** (candidate class) để chỉ tập hợp những lớp đầu tiên được tìm ra cho hệ thống. Sau đây là một số gợi ý để tìm các lớp ứng viên:
 - **Lớp ứng viên** thường là các danh từ được trích ra từ các kịch bản. Ví dụ, các từ hệ thống *System*, tín chỉ *Credit*, môn học *Subject*... là các danh từ nhưng với chúng ta thì hệ thống *System* chỉ là một ranh giới của phần phát triển và bên ngoài nên không thể trở thành lớp ứng viên.
 - **Tác nhân** (Actor), ví dụ sinh viên *Student* hoặc giảng viên *Instructor*, thành viên *Member*.
 - **Biên** (Boundaries), ví dụ giao diện *Interface* liên quan đến các đối tượng nghiệp vụ thực hiện hành động như nhập thông tin môn học cùng với các thông tin về sở thích và hành vi.
 - Các kiểu thông dụng như *String*, *Int*, *Float*... thường được cung cấp bởi ngôn ngữ cài đặt hoặc từ thư viện của ngôn ngữ lập trình.

| 4.2.2 Xác định quan hệ giữa các lớp

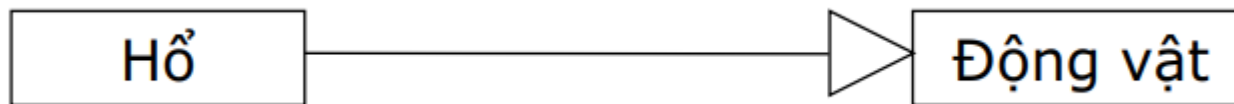
- Với một danh sách các lớp ứng viên, chúng ta sẽ xem xét các quan hệ giữa chúng. Có 4 loại quan hệ chính sau đây:
 - **Kế thừa** (Inheritance): một lớp con sẽ kế thừa tất cả các thuộc tính và hành vi của lớp cha của nó.
 - **Liên kết** (Association): các đối tượng của một lớp được liên kết với các đối tượng của lớp khác.
 - **Kết hợp** (Aggregation): một loại liên kết mạnh, nghĩa là một thể hiện của một lớp được tạo ra bởi các thể hiện của một lớp khác.
 - **Hợp thành** (Composition): quan hệ kết hợp mạnh, nghĩa là một đối tượng hợp thành không thể được dùng chung với các đối tượng khác và nó sẽ mất cùng với đối tượng được hợp thành.
- Kế thừa là một kiểu quan hệ khác với ba loại quan hệ kia. Kế thừa mô tả một quan hệ trong thời gian biên dịch giữa các lớp trong khi các quan hệ kia mô tả kết nối trong thời gian chạy giữa các đối tượng.
- Lựa chọn giữa các quan hệ đòi hỏi một sự khéo léo, nghĩa là cần sử dụng khả năng trực giác, kinh nghiệm và sự phỏng đoán.

| Khái quát hóa và cụ thể hóa

- **Khái quát hoá** (generalization) là sự rút ra các đặc điểm chung của nhiều lớp để tạo thành một lớp giản lược hơn.
 - Được gọi là lớp cha (superclass).
- Ngược lại, **cụ thể hoá** (specialization) là sự tăng cường (bổ sung) thêm một số đặc điểm mới từ một lớp đã cho, tạo thành một lớp cụ thể hơn.
 - Được gọi là lớp con (subclass).
- Một lớp có thể không có lớp cha, có một hay nhiều lớp cha
 - Một lớp chỉ có một lớp cha gọi là lớp thừa kế đơn (simple inheritance)
 - Một lớp có nhiều lớp cha được gọi là lớp thừa kế bội (multiple inheritance)
- Một lớp không có lớp cha và có lớp con, thì được gọi là lớp gốc (lớp cơ sở).
- Thông qua sự khái quát hoá, ta có thể làm xuất hiện các **lớp trừu tượng** (abstract class) là các lớp không có đối tượng cá thể (no instances), mà chỉ dùng để mô tả các đặc điểm chung của những lớp con (lớp dưới) mà thôi.

| Khái quát hóa và cụ thể hóa

- Một lớp trừu tượng thường chứa các **thao tác trừu tượng** (abstract method) là các thao tác chỉ có tiêu đề (tên) mà không có cài đặt
 - Thao tác trừu tượng phải được định nghĩa lại và kèm cài đặt ở các lớp con.
 - Tên của lớp trừu tượng và tiêu đề của thao tác trừu tượng phải được viết nghiêng và có thể kèm thêm xâu tính chất **{abstract}**
- Thuật ngữ kế thừa (inherit) được sử dụng phổ biến trong các ngôn ngữ lập trình, nhằm diễn tả một lớp con có mọi thuộc tính, thao tác và liên kết được mô tả ở một lớp cha
- Lớp con có thể có thêm (riêng) các thuộc tính, các thao tác và các liên kết mới.
- Lớp con có thể định nghĩa lại (overwrite) một thao tác của lớp cha: Sự đa hình (polymorphism).
- Biểu diễn khái quát hóa:



| 4.2.3 Xây dựng biểu đồ lớp

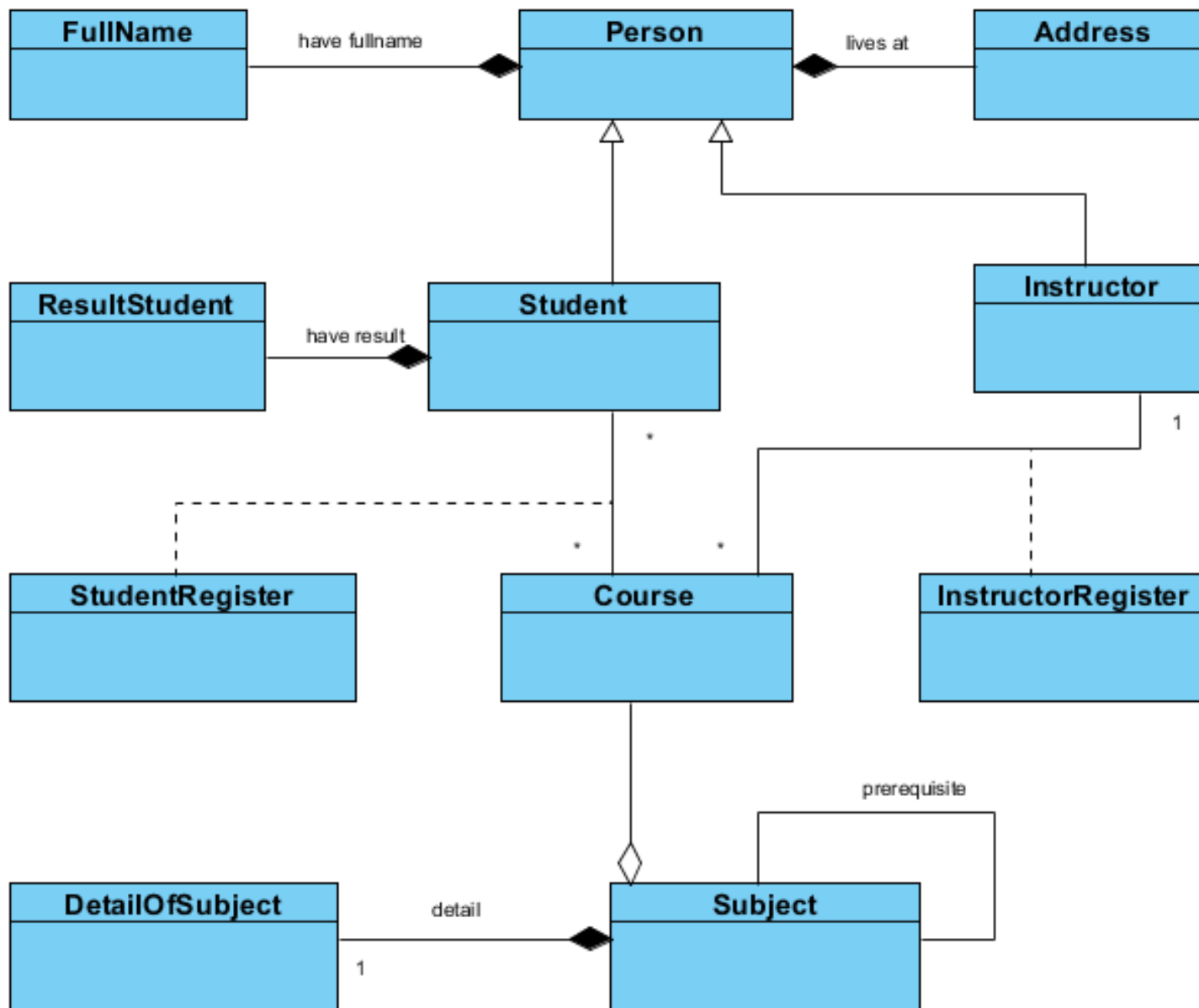
- Biểu đồ lớp biểu diễn các lớp và quan hệ giữa chúng. Biểu đồ lớp cũng thể hiện các thuộc tính và hành động tương ứng với các lớp.
- Mỗi lớp được thể hiện bởi một hình chữ nhật có 3 gian với tên lớp bên trong (in đậm). Nếu lớp thuộc loại trừu tượng, tên lớp in nghiêng hay có thể thêm từ khóa {abstract} bên trên hoặc bên phải lớp thay cho chữ in nghiêng.

Class_Name
attribute1 attribute2 attribute3
method1() method2() method3()

Person
-name : String -birthDate : Date
+getName() : String +setName(name) : void +isBirthday() : boolean

Book
-title : String -authors : String[]
+getTitle() : String +getAuthors() : String[] +addAuthor(name)

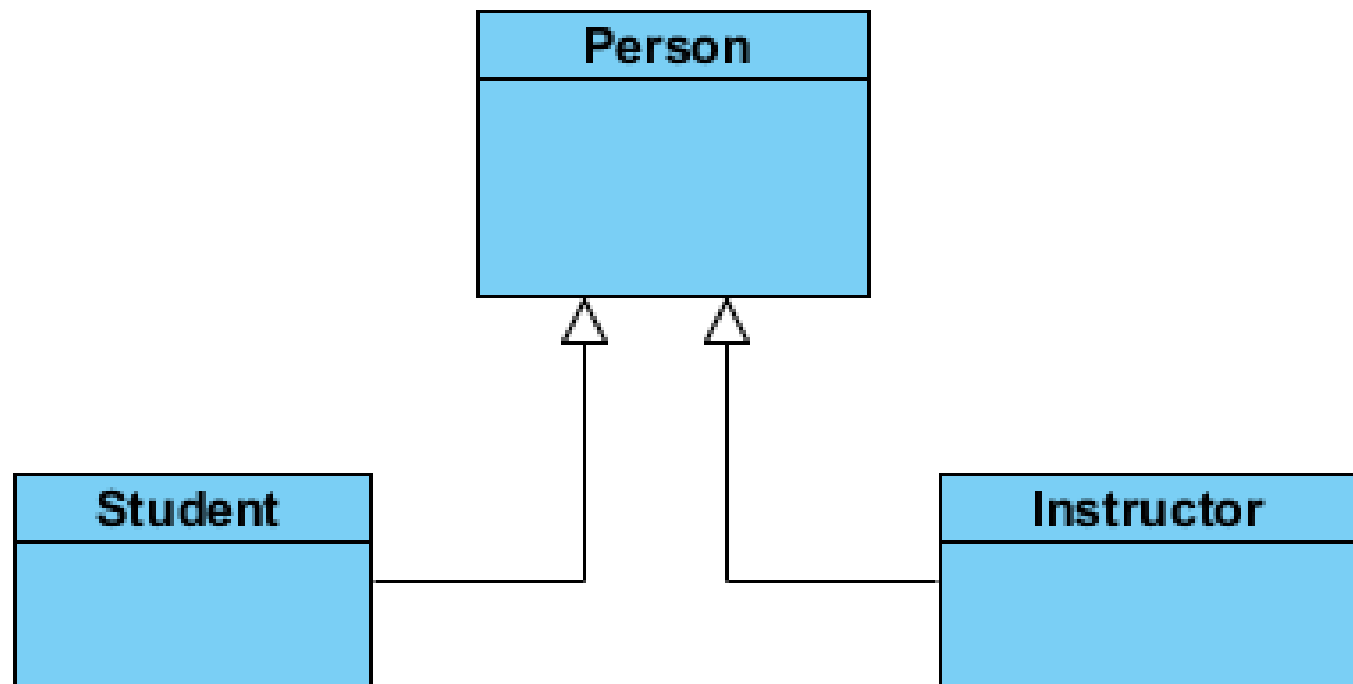
| Biểu đồ lớp Hệ thống quản lý học tập theo tín chỉ



| Biểu diễn các quan hệ giữa các lớp

Quan hệ kế thừa

- Quan hệ kế thừa được mô tả trong biểu đồ lớp bởi một đường với đầu mũi tên màu trắng được vẽ từ lớp con hướng tới lớp cha.
- Ví dụ: Lớp *Student* và *Instructor* là kế thừa của lớp *Person*.



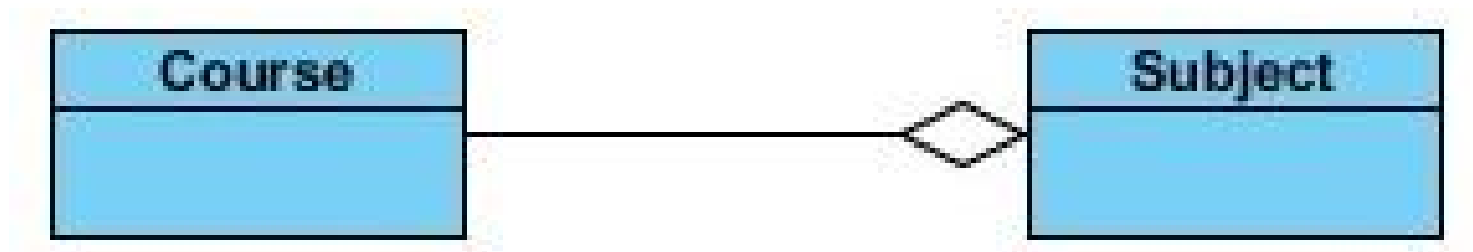
I Quan hệ liên kết

- Quan hệ liên kết được mô tả bởi một đường thẳng nối giữa hai lớp có liên quan về ngữ nghĩa. Ngữ nghĩa đó có thể được mô tả trên đường nối các quan hệ.
- Ví dụ: Ở đây Sinh viên *Student* có quan hệ liên kết với Môn học *Subject*, nghĩa là *Sinh viên* sẽ đăng ký học môn học nào đó. Rõ ràng *Môn Học* không phải là một thành phần của *Sinh viên* và ngược lại.



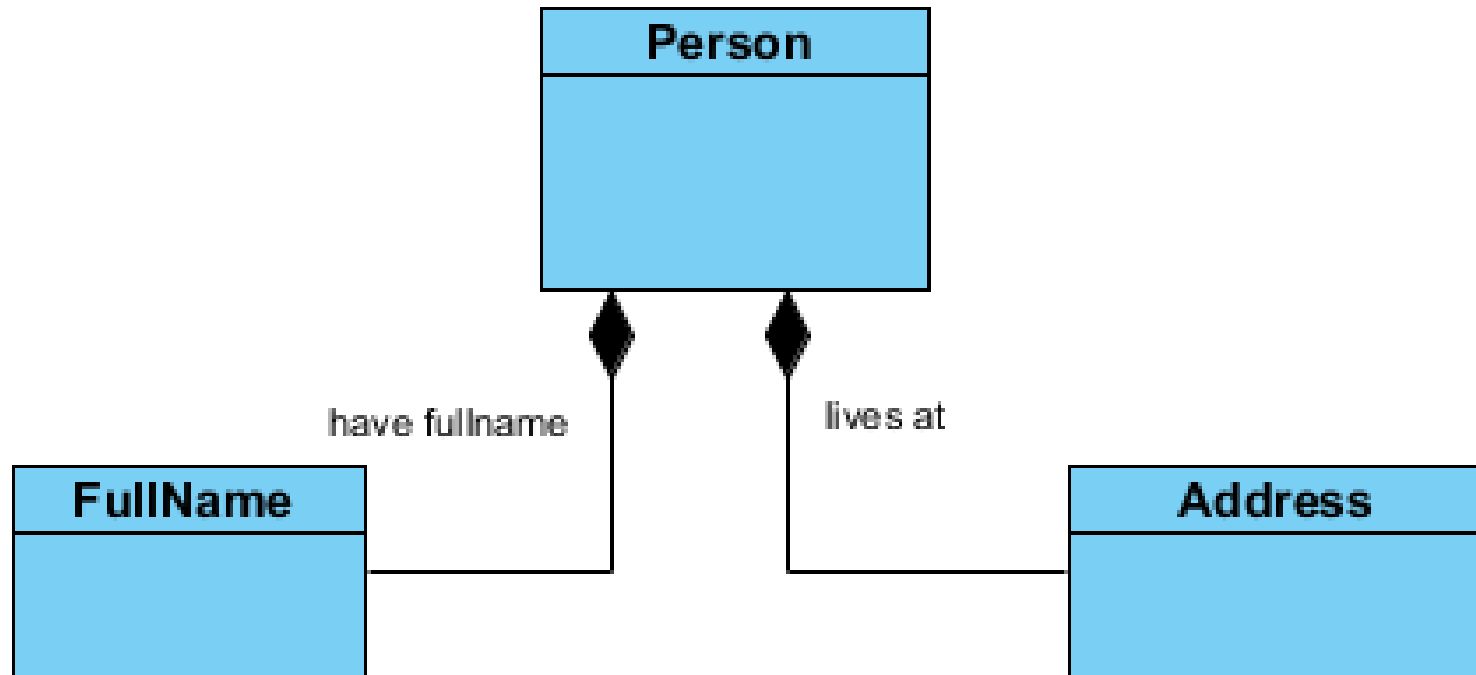
I Quan hệ kết hợp

- Quan hệ kết hợp trong UML được mô tả bởi một đường nối giữa hai lớp với một đầu có hình quả trám trắng ở cuối quan hệ chỉ quan hệ bộ phận trên lớp tổng thể.
- Ví dụ: *Course* là một phần của *Subject*.



I Quan hệ hợp thành

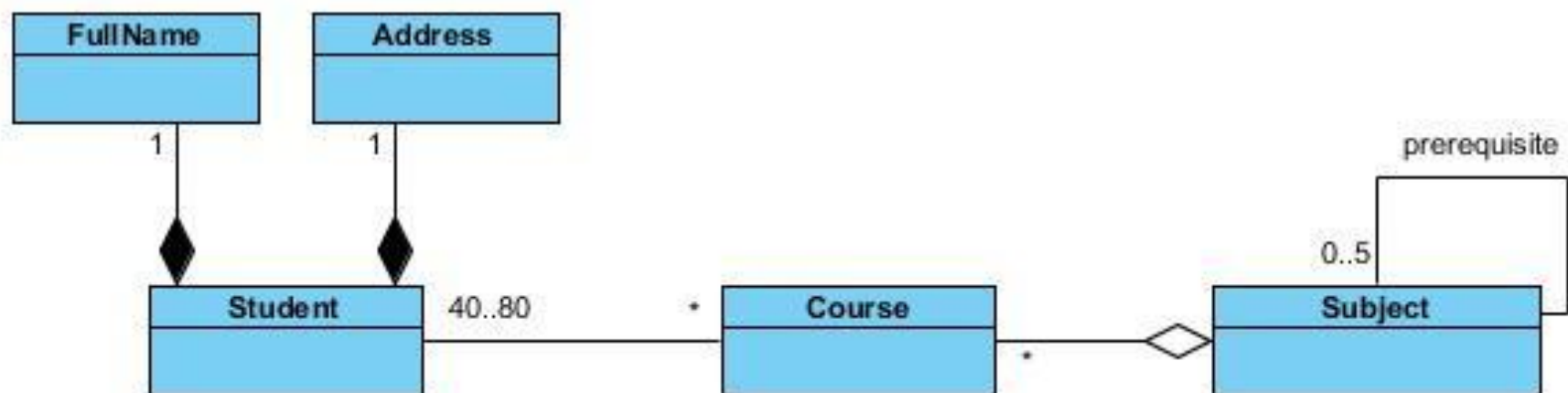
- Quan hệ hợp thành trong UML tương tự như quan hệ kết hợp, nhưng đầu hình quả trám là màu đen ở cuối.
- Ví dụ: *FullName* và *Address* luôn là một phần của *Person*.



| Tính nhiều (Multiplicity)

- Ngoại trừ quan hệ kế thừa, tất cả các quan hệ đều có thể hiện số lượng đối tượng được phép tham gia vào các quan hệ tại cuối mỗi liên kết.
- Số lượng đối tượng có thể có các dạng sau đây:
 - n : lấy chính xác n
 - $m\dots n$: số nào trong dải từ m đến n
 - $p\dots*$: số nào trong dải từ p đến vô cùng
 - $*$: viết gọn của $0\dots*$
 - $0\dots 1$: tùy chọn
- Với quan hệ hợp thành, tính nhiều tại cuối hợp thành luôn luôn là 1.

| Tính nhiều (Multiplicity)



Một Student có duy nhất một FullName và một Address

Một Student có thể tham gia học nhiều Course

Một Course chỉ được học bởi tối thiểu 40 Student hoặc tối đa 80 Student;

Một Course sẽ thuộc về một Subject nhất định

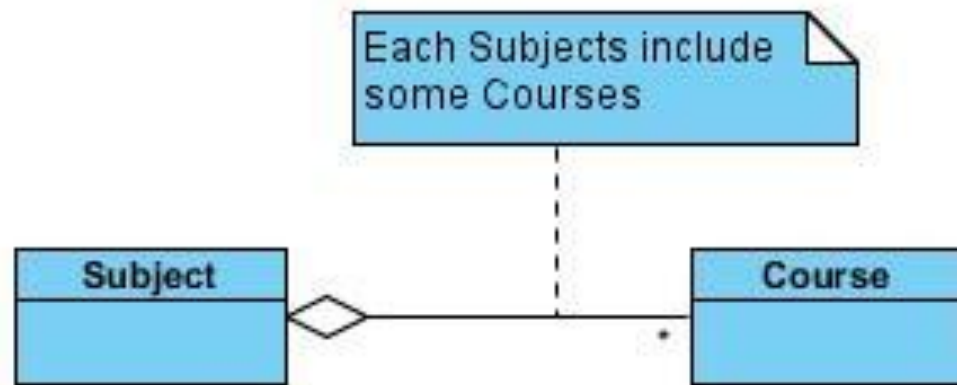
Một Subject có thể có nhiều Course (Course 3 đvht hay 4 đvht)

Một Subject có thể không có môn điều kiện nào (Prerequisite Subject)

Một Subject có thể có tối đa 5 môn học điều kiện

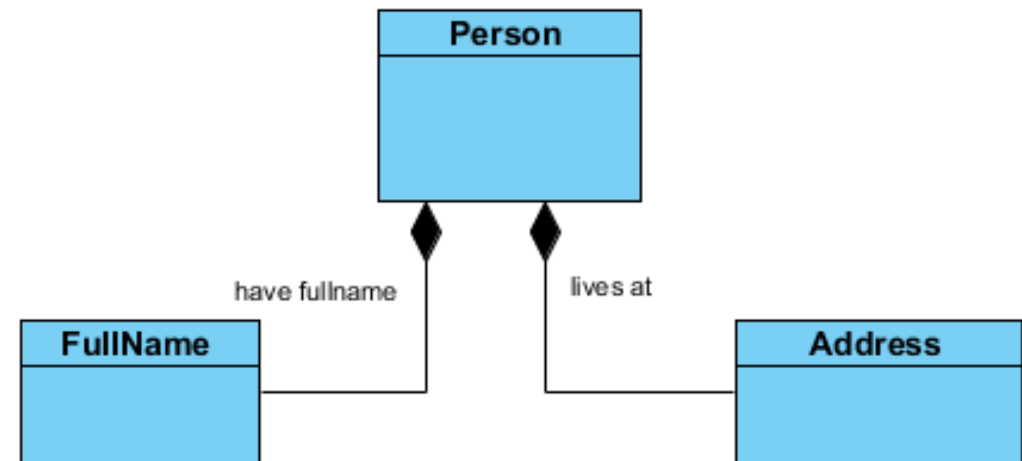
| Các nhãn, vai trò và chú thích cho quan hệ

- Tất cả các quan hệ, ngoại trừ kế thừa, đều có thể sinh ra một **nhãn liên kết**, nhằm chỉ ra bản chất của liên kết.
- Trong các quan hệ, chúng ta có thể biểu diễn các **vai trò** (role) nhằm mô tả nhãn gần đối tượng thể hiện vai trò đó.



Nhãn liên kết

Course là một phần của *Subject*, một *Subject* sẽ bao gồm nhiều *Course* như là thành phần của nó.

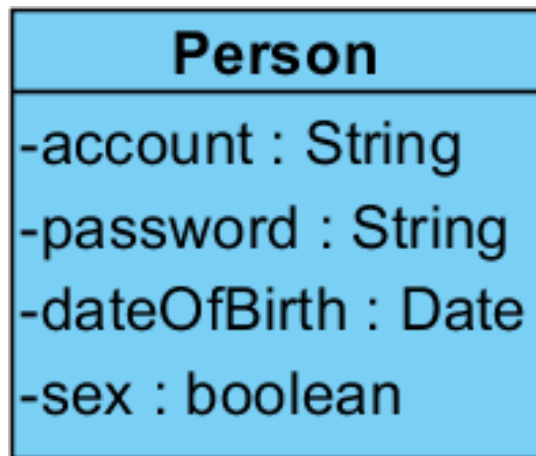


Vai trò

Con người *Person* sống tại *live at* địa chỉ *Address*.

| 4.2.4 Xác định thuộc tính lớp

- Một **thuộc tính** là một *đặc tính* hay *tính chất* của một đối tượng, như là kích cỡ, vị trí, tên, giá, sở thích, giới tính hay bất kì điều gì.
- Trong UML, mỗi thuộc tính có thể được cho bởi kiểu (type) được xác định bởi một lớp hay kiểu cơ bản. Kiểu được biểu diễn sau dấu “:” và đặt sau tên thuộc tính.
- Thuộc tính có thể được hiển thị trong biểu đồ lớp bằng cách thêm một ngăn ở dưới tên lớp.
- Để tiết kiệm không gian, chúng ta có thể viết tài liệu riêng biệt cho chúng thay vì một danh sách thuộc tính hoàn chỉnh với các mô tả.



| Cách xác định các khái niệm

- Đọc tài liệu mô tả hệ thống (phát biểu yêu cầu)
 - Các **danh từ** sẽ có thể là **đối tượng** hay **thuộc tính**,
 - Các **động từ** sẽ có thể là các **thao tác**, **phương thức**.
- Dựa vào đó, xác định các đối tượng sau đây:
 - Các **thực thể** (ví dụ: xe đạp, máy bay, cảm biến, ...)
 - Các **vai trò** (ví dụ: làm mẹ, giảng dạy, giám sát, ...)
 - Các **sự kiện** (ví dụ: hạ cánh, ngắt, đăng ký xe máy, ...)
 - Các **tương tác** (ví dụ: cho vay, thảo luận, ...)
 - Các **tổ chức** (ví dụ: công ty, khoa, lớp, ...)

| Biểu diễn lớp và các thuộc tính

Person
-account : String -password : String -dateOfBirth : Date -sex : boolean

FullName
-firstName : String -midName : String -lastName : String

Address
-section : String -road : String -town : String -district : String -city : String

Subject
-subjectID : String -subName : String -credit : int -faculty : String -major : String

Student
-studentID : String -faculty : String -major : String -class : String -course : ArrayList<Course >

Instructor
-instructorID : String -faculty : String -subjectGroup : String -rank : String -email : String

Course
-courseID : String -startDate : Date -time : String -room : String -capacity : int -student : ArrayList<Student> -examDate : Date -examTime : String -examRoom : String

DetailOfSubject
-subjectID : String -classHourTheory : String -classHourPractice : String -description : String

ResultStudent
-scoreSubject : ArrayList<Subject> -averageScore : float -performance : String -conduct : String -subjectFailed : ArrayList<Subject>

StudentRegister
-semester : String -schoolYear : int -finalScore : float

InstructorRegister
-iID : String -semester : String -schoolYear : int -course : ArrayList<Course>

| Thuộc tính hay quan hệ

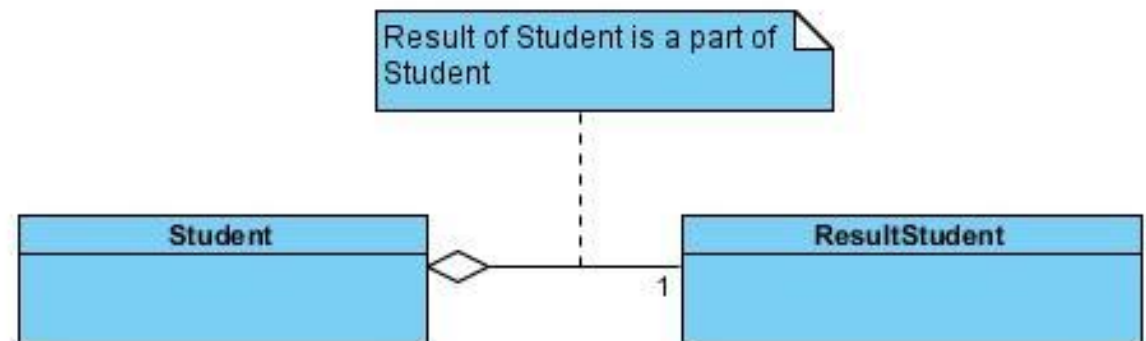
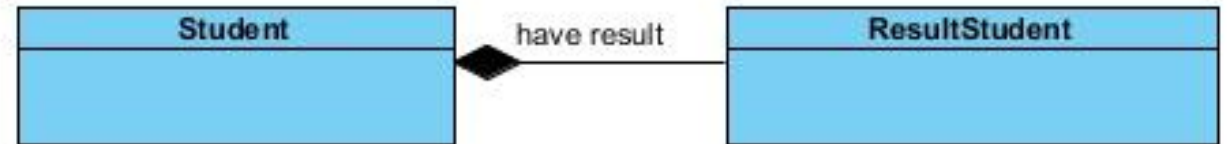
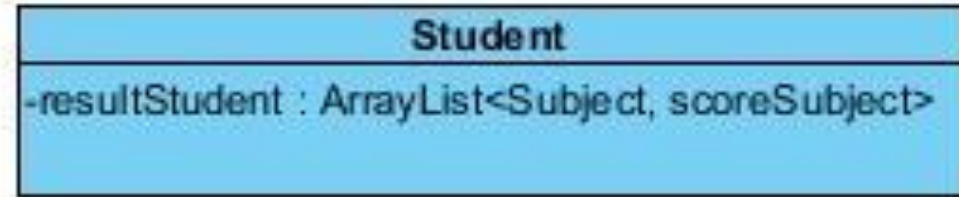
- Thông thường chúng ta cần phải đưa ra những lựa chọn khác nhau cho mô hình hóa thông tin, chọn lựa thiết kế là thuộc tính của lớp hay tách ra thành quan hệ.
- Một lưu ý khi phân tích là cần phải lựa chọn những trường hợp biểu diễn thoả mãn với ngữ cảnh hiện tại nhất nhưng không phải bao giờ ta cũng có một câu trả lời chính xác duy nhất.
- Ví dụ, mô hình kết quả học tập của sinh viên như thế nào được cho là phù hợp nhất với hệ thống?
 - Thêm một thuộc tính cho lớp *Student* gọi là *resultStudent* với kiểu mảng chứa danh sách môn học và kết quả môn học sinh viên đăng ký.
 - Quan hệ kiểu hợp thành giữa *Student* và *ResultStudent*
 - Quan hệ kiểu liên kết giữa *Student* và *ResultStudent*.

| Thuộc tính hay quan hệ

Lựa chọn 1: có thể dẫn đến xây dựng một cơ sở dữ liệu phức tạp cho Student.

Lựa chọn 2: khá hợp lý khi *ResultStudent* là một phần của *Student* và sự tồn tại của lớp *Student* điều khiển sự tồn tại của *ResultStudent*.

Lựa chọn 3: *ResultStudent* vẫn là một thành phần của *Student* nhưng dường như không chính xác khi cho rằng *ResultStudent* có thể tồn tại độc lập với *Student*.

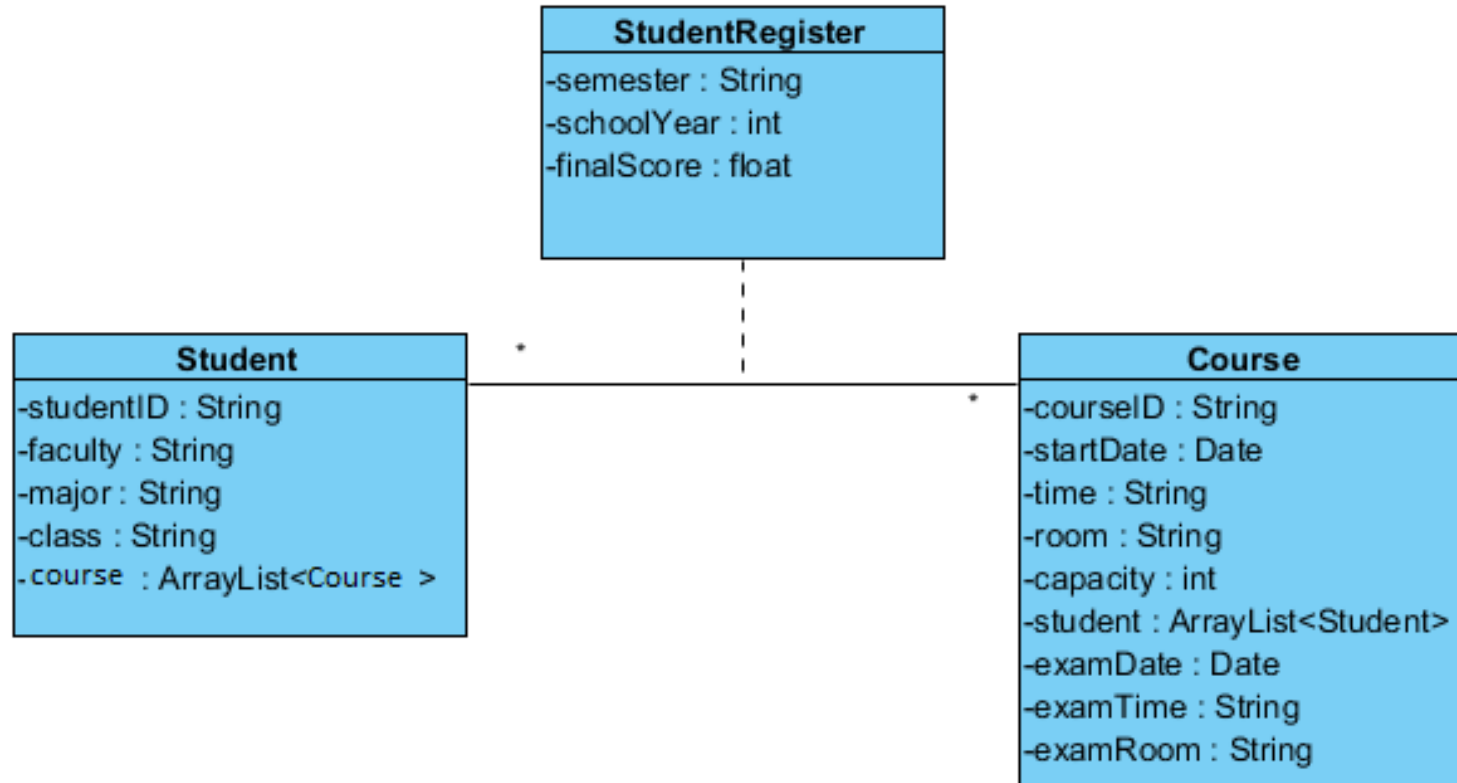


| Thuộc tính của lớp liên kết

- Một lớp được đính kèm trong quan hệ liên kết được gọi là một **lớp liên kết** (association class).
- Một lớp liên kết không được nối tới bất kì lớp nào của quan hệ mà nối tới chính bản thân quan hệ. Cũng giống như một lớp bình thường, lớp liên kết có thể có *thuộc tính*, *phương thức* và *các quan hệ khác*. Lớp liên kết được sử dụng để bổ sung thêm thông tin cho quan hệ liên kết.
- Lớp liên kết mô tả các thuộc tính và hành động của thực thể và chỉ tồn tại khi sự liên kết tồn tại: các thuộc tính và các hành động không liên quan đến các đối tượng trong liên kết.

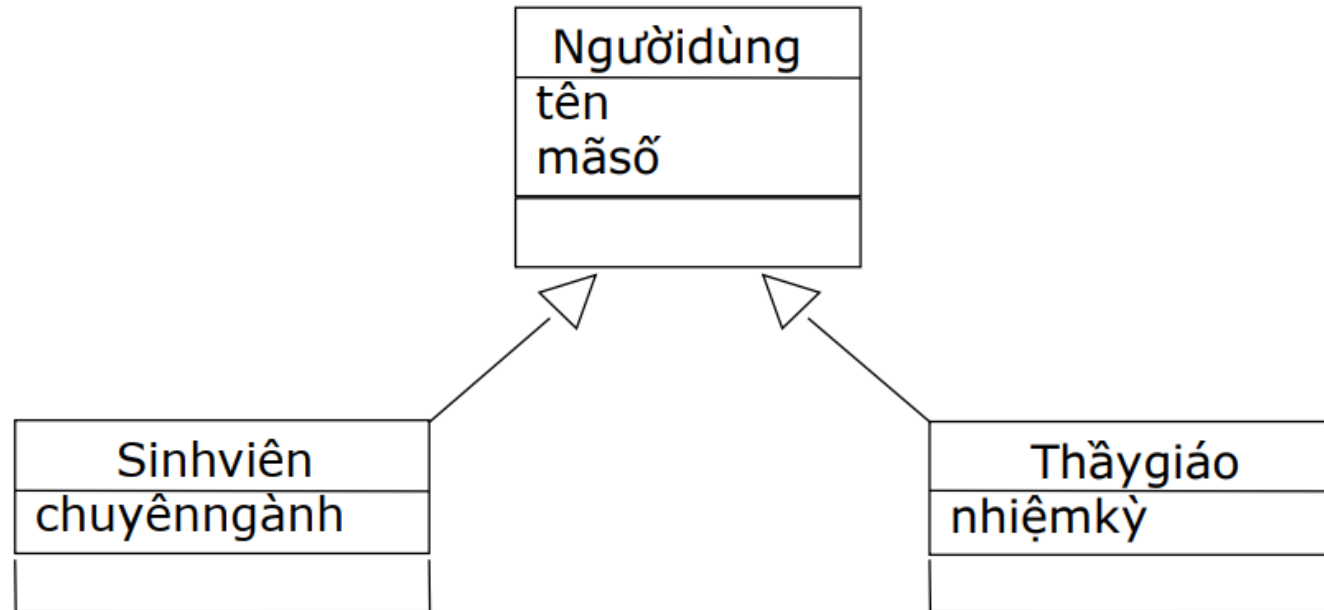
| Thuộc tính của lớp liên kết

- Ví dụ: Lớp *Student* có thể được kết nối với một số bất kỳ đối tượng *Course* và ngược lại.
- Khi *Student* đăng ký học một *Course*, một liên kết mới được tạo trong thời gian chạy giữa *Student* và *Course* tương ứng. Khi đó, một số thuộc tính mới nảy sinh như điểm kết thúc môn học *finalScore*, năm học, học kì.



| Khái quát hóa các lớp

- Để cải thiện tối ưu mô hình biểu diễn lớp, ta tìm cách rút ra các phần chung giữa các lớp để lập thành lớp khái quát hơn.
- Chẳng hạn các lớp “Sinhvien” và “Thầygiáo” có những thuộc tính chung (ví dụ: tên, mã số, ...) => có thể xác lập một lớp khái quát hóa hơn (ví dụ: lớp “Người dùng”).



| 4.3 Phân tích động

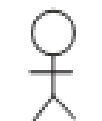
- Mục đích của phân tích động là:
 - Nhằm khẳng định rằng biểu đồ lớp của chúng ta đã hoàn thiện và chính xác để có thể cố định nó sớm hơn. Lý do là vì điều này liên quan đến việc thêm, xóa, sửa các lớp, các quan hệ, các thuộc tính và các thao tác trong các lớp.
 - Để các nhà phát triển lẫn khách hàng có thể tin được rằng tiến trình mô hình hóa hệ thống của chúng ta đến nay có thể cài đặt thành phần mềm.
 - Để kiểm định tính năng của giao diện người dùng sẽ xuất hiện trong hệ thống cuối cùng. Đây là một ý tưởng tốt để phân quyền truy cập hệ thống thành các giao diện tách biệt theo các dòng kịch bản trước khi đi tiến hành thiết kế chi tiết.
- Phần quan trọng nhất của phân tích động là *hiện thực hóa ca sử dụng* (use case realization) nghĩa là làm cho các ca sử dụng trở nên hiện thực hơn để cài đặt sau này bằng thể hiện cách cài đặt hệ thống như tập các đối tượng tương tác với nhau. Gồm các bước sau:
 - Duyệt qua các ca sử dụng của hệ thống, mô phỏng các thông điệp được gửi giữa các đối tượng và ghi lại các kết quả bằng biểu đồ giao tiếp hoặc biểu đồ tuần tự.
 - Gán các thao tác cho các lớp tương ứng.
 - Thêm các lớp để biểu diễn các lớp biên (các giao diện hệ thống) và các lớp điều khiển khi cần thiết.

| 4.3.1 Xây dựng biểu đồ giao tiếp

- Khi mô phỏng các thông điệp được gửi đi giữa các đối tượng trong pha phân tích, chúng ta cần ghi lại các kết quả tương tác này bằng **biểu đồ giao tiếp** hay **biểu đồ tuần tự** trong UML.
- Mặc dù các thông tin thể hiện trên hai biểu đồ này được xem là tương tự nhau, nhưng biểu đồ giao tiếp được xem là tốt hơn cho hiện thực hóa ca sử dụng và dễ dàng xây dựng hơn.
- Trong pha **xác định yêu cầu**, chúng ta có đề cập xây dựng **biểu đồ giao tiếp** nhưng là tùy chọn và có biểu diễn đơn giản hơn nhằm mô tả hoạt động nghiệp vụ hiện thời của khách hàng. Trong khi đó, biểu đồ giao tiếp mức phân tích thể hiện đầy đủ hơn và bao gồm các thành phần:
 - Các tác nhân tương tác với các *đối tượng biên*.
 - Các *đối tượng biên* tương tác với các *đối tượng bên trong* hệ thống.
 - Các *đối tượng bên trong* hệ thống tương tác với các *đối tượng biên* của hệ thống bên ngoài.
 - Các thông điệp thể hiện như *kiểu phương thức* để có thể gán cho các đối tượng tương ứng sau này.

| Các đối tượng biên, thực thể và điều khiển

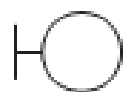
- **Tác nhân:** Là một người hoặc hệ thống đang tồn tại bên ngoài hệ thống của chúng ta.
- **Đối tượng biên:** một đối tượng nằm tại biên hệ thống nghĩa là giữa hệ thống và tác nhân. Với tác nhân là con người, đối tượng biên là giao diện có nhiệm vụ nhận lệnh, truy vấn và hiển thị kết quả. Mỗi đối tượng biên thường tương ứng với một ca sử dụng hay nhóm ca sử dụng có liên quan nhau. Các đối tượng biên như vậy thường tương ứng với phác họa giao diện.
- **Đối tượng thực thể:** Một đối tượng nằm bên trong hệ thống, mô tả các khái niệm nghiệp vụ như Sinh viên *Student*, Khách hàng *Customer*, Môn học *Subject*... và chứa các thông tin về nghiệp vụ cần xử lý.
- **Đối tượng điều khiển:** Một đối tượng bên trong hệ thống đóng gói một tiến trình phức tạp. Đối tượng điều khiển thường là một đối tượng dịch vụ để cung cấp các dịch vụ như: điều khiển tất cả hoặc một phần của tiến trình hệ thống, tạo các thực thể mới, triệu gọi các thực thể đang tồn tại.



Actor



Entity



Boundary

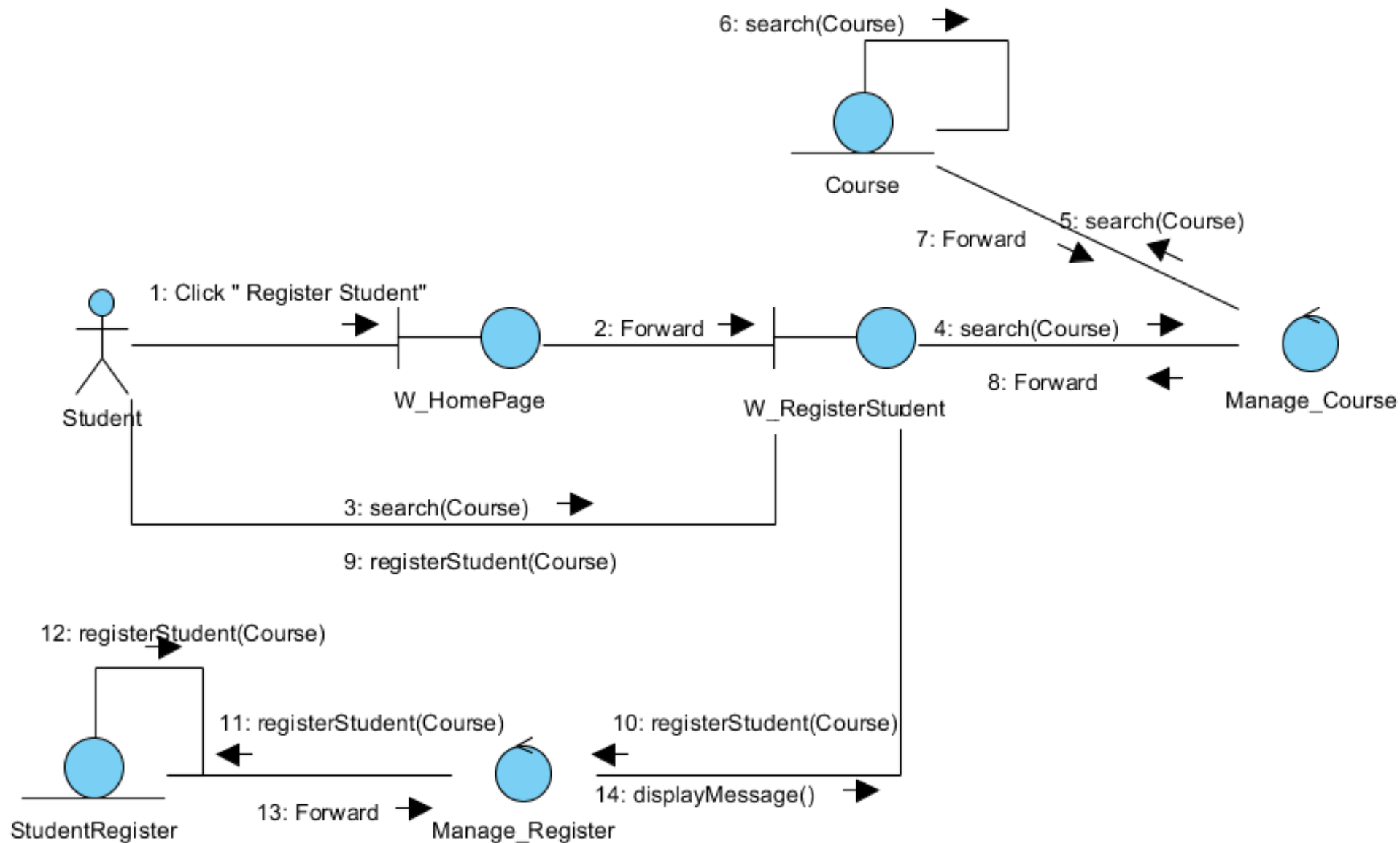


Controller

| Các thành phần của biểu đồ giao tiếp

- **Tác nhân:** được biểu diễn giống trong biểu đồ ca sử dụng.
- **Đối tượng:** được biểu diễn và gán tên.
- Một đường thẳng nối giữa các đối tượng chỉ ra thông điệp được gửi đến đâu.
- Thông điệp được thể hiện với số tuần tự để chỉ ra vị trí thông điệp trong giao tiếp, tên thông điệp và danh sách tham số để trong dấu ngoặc.
- Mũi tên chỉ hướng thông điệp được gửi đi.
- Nhãn được sử dụng để định danh cho đối tượng và tham số như *tên:kiểu*, ví dụ s: *Student*.
- Gán giá trị trả về cho một cái tên và được biểu diễn như sau: *n = getNumber();*
- Thông điệp có điều kiện: biểu diễn: *[điều kiện]thông điệp*.
- Lặp được thể hiện bằng ký hiệu * và theo sau là số lần lặp. Ví dụ, *3 (có nghĩa là lặp 3 lần).

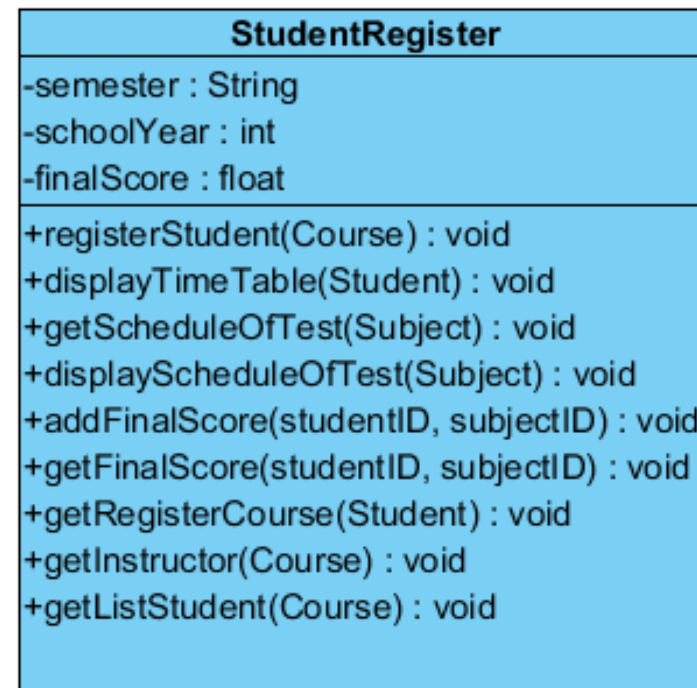
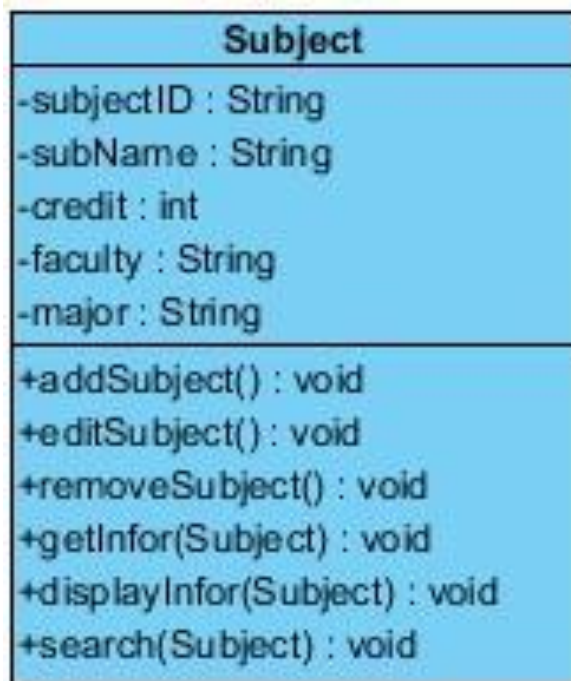
| Biểu đồ giao tiếp cho Sinh viên đăng ký học



| 4.3.2 Gán phương thức cho các lớp

- Mỗi thông điệp trong **biểu đồ giao tiếp** tương ứng với một **phương thức** trên một lớp, chính vì vậy chúng ta nên ghi lại các phương thức để có một tập hoàn thiện các hiện thực hóa ca sử dụng.
- Dạng tổng quát của cách biểu diễn phương thức trong UML, trong đó các tên và kiểu tham số, kiểu trả về là không bắt buộc:

OperationName(paramName1:paramType1,paramName2:paramType2):ReturnType



| Gán phương thức cho lớp dựa trên trách nhiệm (Responsibility)

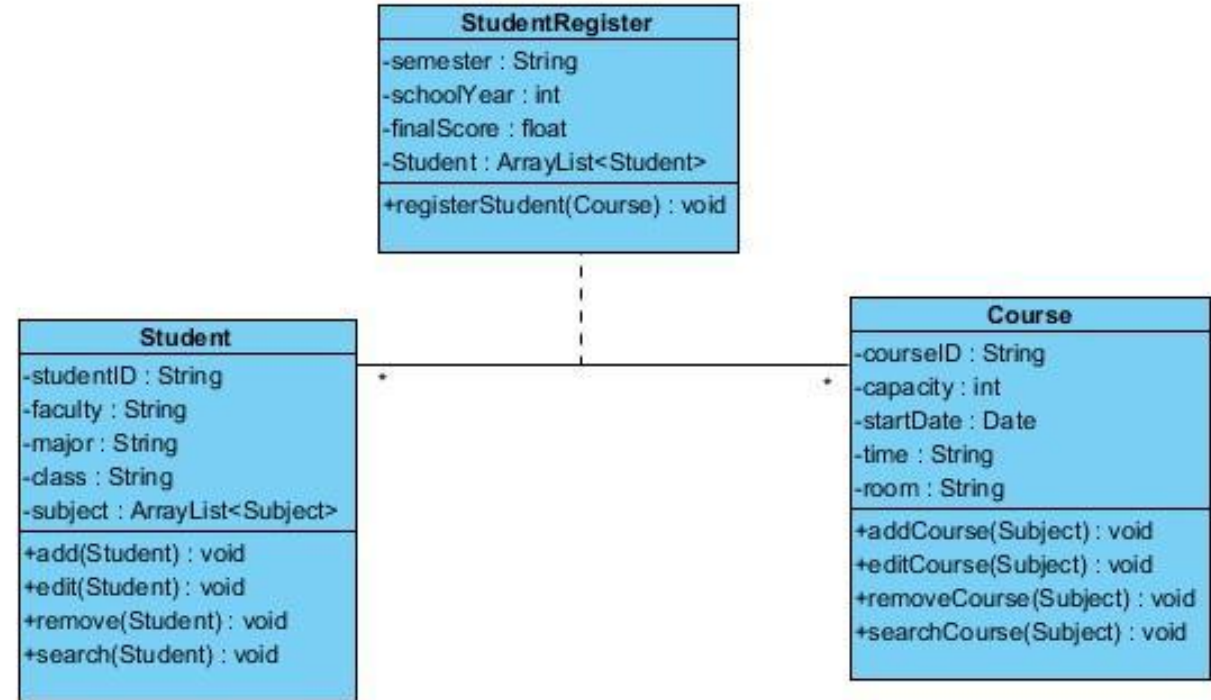
- Câu hỏi đặt ra là **một phương thức nào đó nên được gán cho lớp nào?** Bất cứ khi nào thêm chi tiết vào đối tượng, ta cần xem xét đến trách nhiệm của đối tượng đó với hệ thống.
 - Khi phát hiện thông tin hoặc hành động nào đó cần cho hệ thống hãy tự hỏi rằng “đối tượng nào chịu trách nhiệm thực hiện hành động này?” **Thông tin cho ta các thuộc tính, hành động cho ta các thao tác.**
 - Trong việc gán phương thức, nguyên lý **kết hợp mạnh** (strong cohesion) cho rằng một đối tượng không nên chịu trách nhiệm nhiều hơn một thao tác trong một luồng công việc.
 - Ví dụ, nếu phải chịu trách nhiệm lấy các thông tin chi tiết từ lớp Student và xử lý nó, chúng ta nên chọn hai đối tượng để thực hiện hai công việc này, một là đối tượng biên và một đối tượng điều khiển.
- ➔ Hai đối tượng này phải kết hợp chặt chẽ với nhau để tạo ra được một luồng xử lý công việc.

| Gán phương thức cho lớp dựa trên trách nhiệm (Responsibility)

- Thuật ngữ **kết dính** (coupling) nhằm chỉ mức độ phụ thuộc giữa các lớp với nhau và hai lớp phụ thuộc mạnh vào nhau được gọi là có *kết dính chặt* (tightly/highly coupled), ngược lại gọi là *kết dính lỏng lẻo* (loosely coupled).
- Nguyên lý **kết dính lỏng lẻo** (loosely coupled) cho rằng nên xem các đối tượng như là một khách hàng (yêu cầu các câu hỏi và đưa ra câu lệnh) hoặc nhà cung cấp (trả lời các câu hỏi và tiến hành các dịch vụ) hơn là một đối tượng cộng tác cả hai chiều. Khi đó, nhà cung cấp gọi là kết dính lỏng lẻo với các khách hàng của nó.

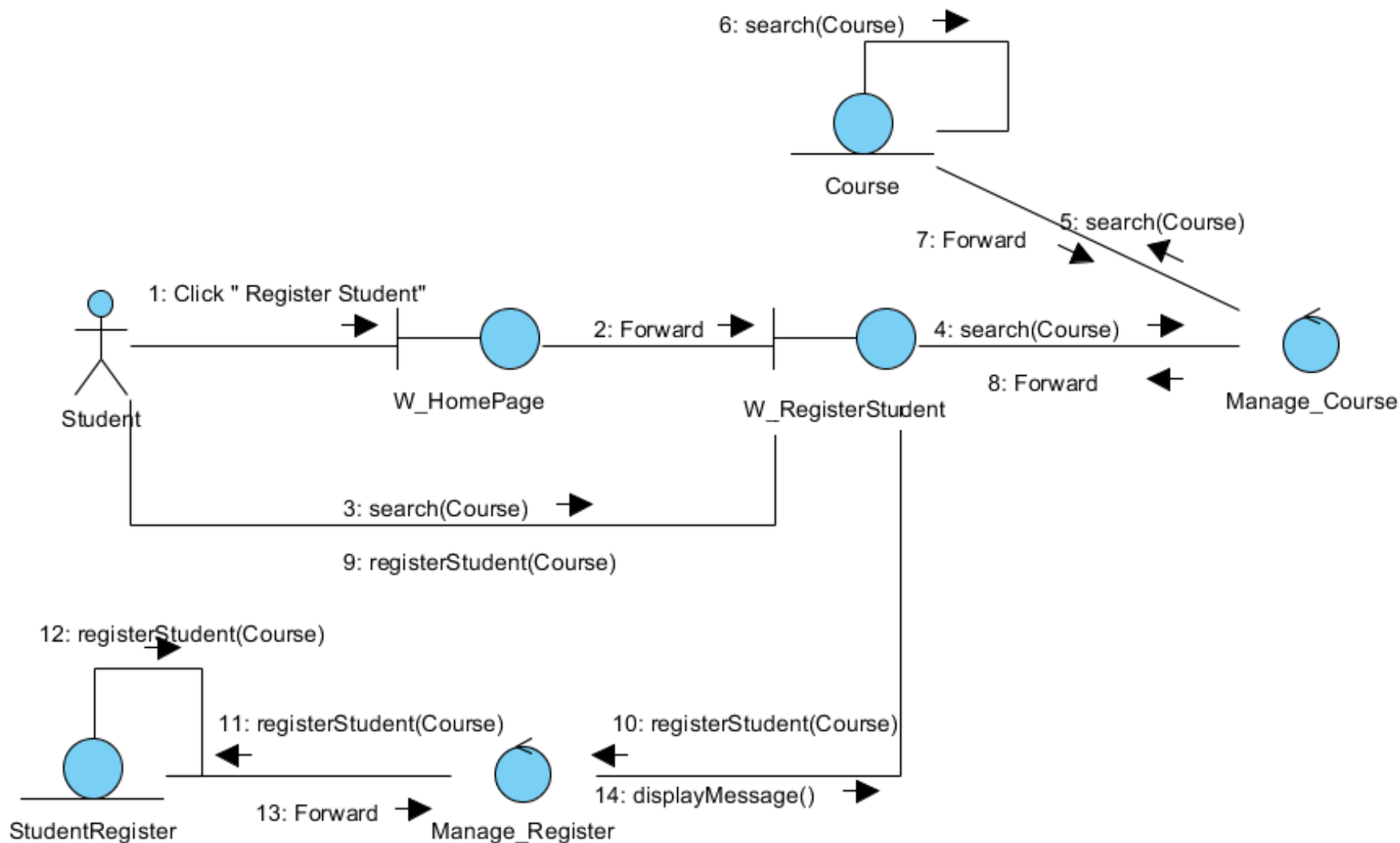
I Hệ thống Quản lý học tập theo tín chỉ

- Phương thức *registerStudent(Course)* thuộc lớp *RegisterStudent* là lớp liên kết giữa *Student* và *Course*.
- Phương thức *registerStudent(Course)* thuộc lớp *RegisterStudent* có thể đảm bảo tính *kết dính lỏng lẻo* trong thiết kế quan hệ giữa các lớp.
- Các phương thức thuộc lớp *Student*, *Course* đều gọi đến các thuộc tính đã được khai báo trước đó trong các lớp tương ứng nên không ảnh hưởng đến lớp *RegisterStudent*.



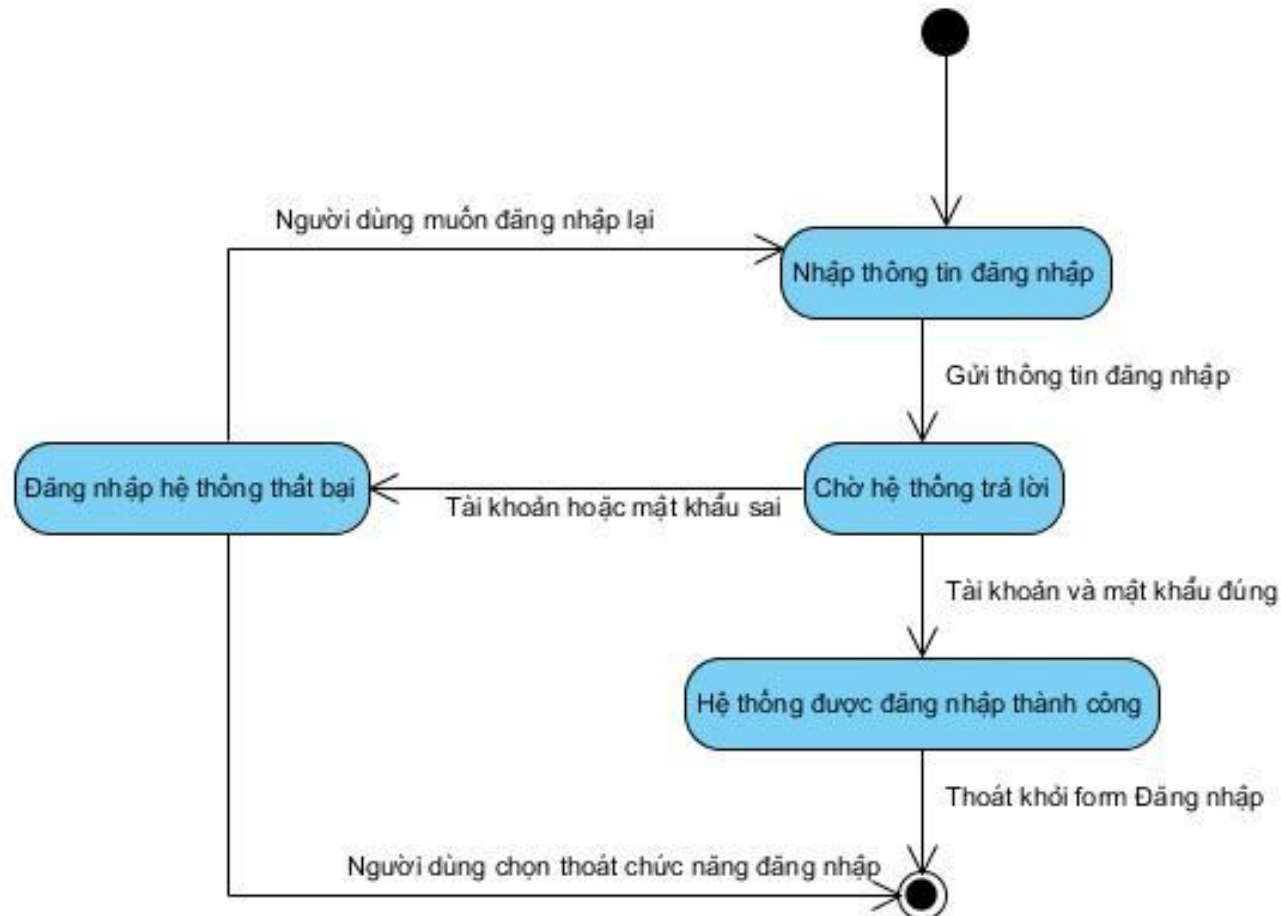
| Gán phương thức cho các lớp dựa trên Biểu đồ giao tiếp

- RegisterStudent
 - registerStudent(course)
 - forward()
- ManageRegister
 - displayMessage()
 - registerStudent(course)
- W_RegisterStudent
 - registerStudent(course)
 - search(course)
- ManageCourse
 - forward()
 - search(course)
- Course
 - search(course)
 - forward()



| Mô hình hóa trạng thái

- Đôi khi, một thực thể có một vòng đời phức tạp nên cần biểu diễn bằng biểu đồ máy trạng thái. Biểu đồ này hỗ trợ cho chúng ta khi xác định các phương thức với các tham số cho hành vi của chính đối tượng đó.



I Tổng kết

- Để tiến hành phân tích hệ thống chúng ta cần nắm rõ Đầu vào, Đầu ra của quá trình này. Trong đó, đầu vào của quá trình Phân tích là sản phẩm của pha Xác định yêu cầu bao gồm:
 - Danh sách các tác nhân
 - Danh sách các ca sử dụng
 - Các kịch bản
 - Biểu đồ ca sử dụng
 - Các phác họa giao diện
- Đầu ra của pha Phân tích là *biểu đồ lớp*, *biểu đồ giao tiếp* và *biểu đồ trạng thái*. Quá trình phân tích bao gồm hai giai đoạn: *phân tích tĩnh* và *phân tích động*.
- Trong phân tích tĩnh, chúng ta tiến hành thực hiện những công việc sau:
 - Xác định các lớp.
 - Xét mối quan hệ giữa các lớp.
 - Tìm các thuộc tính của các lớp.
 - Đưa ra biểu đồ lớp.
- Đối với phân tích động là xây dựng *biểu đồ giao tiếp* và *gán phương thức* cho các lớp.

| Case Study

- Tham khảo Case Study *Hệ quản lý Đăng ký học theo tín chỉ* trong giáo trình (Trang 106).

| Bài tập cá nhân

1. Trong hệ quản lý bán hàng, xây dựng các ca sử dụng **nhập hàng** dành cho nhân viên quản lý kho, **nhập hóa đơn** dành cho nhân viên bán hàng và **đặt hàng** dành cho khách hàng. Hãy xác định **các lớp tương ứng**, xây dựng biểu đồ lớp và các biểu đồ giao tiếp của các ca sử dụng này.
2. Dựa trên các biểu đồ giao tiếp trong câu 1, hãy xác định các phương thức và gán phương thức cho các lớp tương ứng.

